

초등부 2, 중등부 1. 종이접기

풀이 작성자: 이재웅

부분문제 1 ($k = 1$)

직접 모든 경우의 수를 시도해 본 다음 가능한 모든 입력에 대해 답을 저장한 후 출력해주면 된다.

부분문제 2 (U, R 만 존재)

가로와 세로로 접는 방법이 고정되어 있을 경우 접는 순서는 중요하지 않다는 것을 관찰할 수 있고, U 와 R 을 각각 k 번 실행할 때 점의 규칙성을 찾아서 출력해주면 된다.

부분문제 3 ($k \leq 8$)

종이를 접은 순서의 역순으로 펼칠 때마다 뚫린 점의 개수가 2배로 늘어나고 새로 추가되는 점의 위치는 접은 방향과 기존 점의 위치에 의해 결정됨을 알 수 있다. 예를 들어, 0번 위치에 뚫린 점이 있는 종이를 오른쪽으로 펼치면 점은 2배로 늘어나는데, 펼쳐진 종이의 왼쪽에는 0번 점이 그대로 있고 오른쪽에는 1번 점이 있게 된다.

기존의 점과 새로 생긴 점은 접은 선을 기준으로 대칭을 이루므로, 2차원 배열 등을 통해 점의 위치를 기록하면 다음 단계에서 점의 위치 또한 구할 수 있게 된다. 기존 점의 위치와 종이를 펼치는 방향에 따라 새로 생기는 점의 위치를 조건문을 통해 미리 구해놓은 다음, 종이를 차례차례 펼쳐가면서 점들의 변화를 저장한 후 최종 상태를 출력해주면 된다.

초등부 3, 중등부 2. 등산 마니아

풀이 작성자: 구재현

부분문제 1 (그래프가 직선)

두 정수 i, j 를 골랐을 경우 ($i < j$) 다양성이 $j - 1$ 이다. j 를 고정할 경우, 가능한 i 의 개수는 $j - 1$ 이다. 고로 모든 $1 \leq j \leq N$ 에 대해 $(j - 1)^2$ 의 합을 계산하고 출력하면 된다.

부분문제 2 (그래프가 성계)

두 정수 i, j 를 골랐을 경우 ($i < j$) 다양성은 $i = 1$ 일 때 1, 아니면 2이다. $i = 1$ 인 쌍은 $N - 1$ 개 있고, 그렇지 않은 쌍은 $\frac{(N-1)(N-2)}{2}$ 개 있다. 고로 답은 $(N - 1)^2$ 이다.

부분문제 3 (그래프가 완전 이진 트리)

문제의 조건이 복잡하지만, 결국 $N = 2^{d+1} - 1$ 인 깊이 d 의 완전 이진 트리가 주어진다는 뜻이다. i 의 깊이, j 의 깊이, 그리고 1번 정점에서 처음으로 이 두 정점이 갈라지는 위치 k 의 깊이, 이 세 가지를 고정하자. (k 를 보통 i, j 의 *Lowest Common Ancestor (LCA)*) 라고 부른다). i 의 깊이와 j 의 깊이 모두가 k 의 깊이보다 큰 경우, 둘 중 하나만 k 의 깊이보다 큰 경우로 케이스를 나눠서, 가능한 i, j, k 의 경우의 수를 2의 지수승 곱의 수로 계산할 수 있다. $d = O(\log N)$ 이니 $O(d^3) = O(\log^3 N)$ 시간에 문제가 해결된다.

부분문제 4 ($N \leq 100$)

모든 가능한 $i < j$ 를 순회하자. $LCA(i, j)$ 를 두 정점 i, j 의 Lowest Common Ancestor라고 하고, $depth[i]$ 를 i 번 정점과 1번 정점의 거리라고 하면, 답은 $depth[i] + depth[j] - depth[LCA(i, j)]$ 가 된다. 두 정점의 LCA는 $O(N)$ 시간에 계산할 수 있다. 전체 문제가 $O(N^3)$ 시간에 해결된다.

부분문제 5 ($N \leq 5000$)

$dist(i, j)$ 를 두 정점 i, j 의 거리라고 정의하면, 고정된 쌍 $i < j$ 에 대해서 다양성은 $\frac{dist(1, i) + dist(i, j) + dist(j, 1)}{2}$ 이다. 모든 두 정점 간의 거리를 $O(N)$ 번의 깊이 우선 탐색으로 계산해 둘 수 있다. 전체 문제가 $O(N^2)$ 시간에 해결된다.

부분문제 6

각 쌍에 대해서 다양성의 합을 센다는 것은, 각 쌍에 대해서 조건을 만족하는 어떤 간선의 수를 센다는 것이다. 반대로, 모든 간선에 대해서, 이 간선이 특정 쌍의 다양성에 기여하는 조건을 파악하고, 이 간선이 다양성에 기여하는 쌍의 개수를 세어주자.

트리의 루트를 1번 정점으로 두자. v 와 v 의 부모를 잇는 간선에 대해서, v 의 서브트리에 i 혹은 j 정점이 있다면, 이 간선은 1번 정점과 두 정점 중 하나를 잇는 경로 상에 있으니 다양성에 기여한다. 둘 다 없다면, 이 간선이 없다 하더라도 1, i, j 번 정점 사이를 오가는 데는 문제가 없고, 다양성에 기여하지 않는다.

고로, v 와 v 의 부모를 잇는 간선에 대해서 문제의 정답은, 전체 트리에서 두 개의 정점을 고르면서 v 의 서브트리에서 최소 하나의 정점을 뽑는 경우의 수와 동일하다. v 의 서브트리에 속하는 정점의 개수를 $O(N)$ 에 전처리 한다면 이 경우의 수는 간단한 수식으로 셀 수 있다. 전체 문제가 $O(N)$ 에 해결된다.

중등부 3. 버블버블

풀이 작성자: 윤창기

$X = \max(A_1, \dots, A_N)$ 이라고 하자. 모든 풀이에 앞서, 수 하나를 바꿀 때는 1 이상 X 이하의 정수값에 대해서 시도해보면 충분하다는 사실을 알 수 있다. 정수가 아닌 실수로 수를 바꾸는 경우, 가장 가까운 정수로 답을 바꾸어도 교환 횟수가 변하지 않는다.

부분문제 1 ($N \leq 200, X \leq 200$)

수 하나를 바꿀 수 있는 경우가 $O(NX)$ 가지이므로, 모든 경우에 대해 버블 정렬을 직접 시도하는 경우 $O(N^3X)$ 의 시간복잡도로 구현에 따라 점수를 받을 수 있다. 추가적으로 버블 정렬의 교환 횟수는 “ $i < j$ 이지만 $A_i > A_j$ 인 $1 \leq i < j \leq N$ 의 개수”와 같으며, 이를 수열의 inversion count라고 한다. 단순히 inversion count를 계산하는 방법은 $O(N^2)$ 시간이 걸리지만, A_i 를 변경할 때는 i 번째 수와 관련된 inversion에만 변화가 생긴다는 점을 관찰하여 $O(N^2X)$ 에 문제를 해결할 수도 있다.

부분문제 2 ($N \leq 3000, X \leq 3000$)

수열의 inversion count는 다음 질의를 처리할 줄 알면 쉽게 구할 수 있다.

- 현재 집합에서 x 보다 작은 수의 개수를 반환한다. ($1 \leq x \leq X$)
- 집합에 x 를 하나 넣는다. ($1 \leq x \leq X$)

두 질의는 세그먼트 트리, 펜윅 트리 등의 자료구조를 이용하여 매번 $O(\log X)$ 의 시간에 답할 수 있으며, 때문에 $O(N \log X)$ 시간에 수열의 inversion count를 구할 수 있다. 이를 이용하여 $A_i = x$ 일 때 inversion count가 어떻게 바뀌는지를 위 질의를 응용하여 구할 수 있으며, 가능한 모든 경우에 대해 시도함으로써 $O(NX \log X)$ 의 시간복잡도에 문제를 해결할 수 있다.

부분문제 3 ($N \leq 15000, X \leq 15000$)

($1 \leq i \leq N, 1 \leq j \leq X$)에 대해 다음의 두 수열을 정의하자.

- $G_{i,j}$: $A_1, \dots, A_{i-1}$ 중 j 보다 큰 수의 개수
- $L_{i,j}$: $A_{i+1}, \dots, A_n$ 중 j 보다 작은 수의 개수

정의로부터 수열의 inversion count는 $I = \sum_{i=1}^n G_{i,A_i}$ 가 됨을 알 수 있다. 또한, A_i 를 j 로 바꿨을 때 inversion count는 $I - (G_{i,A_i} + L_{i,A_i}) + (G_{i,j} + L_{i,j})$ 가 됨을 알 수 있다. 따라서 G, L 을 이용하여 가능한 $O(NX)$ 가지의 경우마다 $O(1)$ 시간에 inversion count를 계산할 수 있다. 또한, G, L 의 경우 간단한 dynamic programming 방법으로 총 $O(NX)$ 시간에 계산할 수 있다. G, L 을 모두 저장하는 데 많은 메모리가 필요할 수 있으나, 이는 $G_{i,j}$ 를 구할 때 $G_{i-1,j}$ 의 값만 필요하다는 사실을 이용하면 필요한 메모리를 $O(N)$ 으로 줄일 수 있다.

부분문제 4 ($N \leq 300000, X \leq 1000000$)

바꿀 위치 j 를 고정하고, $i < j$ 에 대해 A_i 가 inversion count를 얼마나 늘리는지 생각해보자. A_j 를 $[1, A_i - 1]$ 의 수 중 하나로 바꾸는 경우 1, $[A_i, X]$ 의 수 중 하나로 바꾸는 경우 0만큼 inversion count가 늘어난다. 마찬가지로 $k > j$ 에 대해서는 A_j 가 $[1, A_k]$ 에 들어가도록 바꿀 때 0, $[A_k + 1, A_n]$ 에 들어가도록 바꿀 때 1만큼 inversion count가 늘어난다. 따라서 lazy propagation을 지원하는 최솟값 세그먼트 트리를 이용하여, A_j 를 변경했을 때 inversion count의 최솟값을 $O(N \log X)$ 또는 $O(N \log N)$ 시간에 구할 수 있다.

중등부 4, 고등부 3. 화려한 정사각형

풀이 작성자: 구재현

좌표 범위의 최댓값을 X 라 표기한다.

부분문제 1 ($N, X \leq 50$)

격자 위의 정사각형은 왼쪽 아래 모서리의 x 좌표, y 좌표, 그리고 변의 길이로 표현할 수 있다. 고로 모든 가능한 $O(X^3)$ 개의 정사각형을 나열한 후, 이들 안에 들어있는 점들의 서로 다른 색이 K 개 인지를 배열에 마킹하면 된다. 전체 문제가 $O(X^3N)$ 에 해결된다.

부분문제 2 ($K \leq 50, X \leq 2500$)

왼쪽 아래 모서리의 x 좌표, y 좌표를 고정하자. 각각의 색에 대해서, 해당 색의 점을 포함하기 위해 필요한 변의 길이 최솟값을 계산하고, 이의 최댓값을 변의 길이로 취할 것이다. $DP[X][i][j]$ 를 (i, j) 를 왼쪽 모서리로 하는 정사각형이 색 X 의 점을 포함하기 위해 가져야 하는 최소 변의 길이라고 하자. 색 X 의 점이 (i, j) 위치에 있다면 이는 0이고, 아니면 이는 $\min(DP[X][i+1][j], DP[X][i+1][j+1], DP[X][i][j+1]) + 1$ 이라는 점화식으로 계산 가능하다. 전체 문제가 $O(KX^2)$ 에 해결된다.

부분문제 3 ($K = 2$)

두 점 $(x_1, y_1), (x_2, y_2)$ 를 포함하는 최소 크기 정사각형의 변의 넓이는 $\max(|x_1 - x_2|, |y_1 - y_2|)$ 이다. 이는 두 점의 L_∞ 거리 (체비셰프 거리) 와 동일하다. 색깔 1의 점과 색깔 2의 점을 각각 하나 골라서 위 값을 최소화해야 한다. 즉, 체비셰프 거리 상 가장 가까운 두 점 쌍을 찾는 것이다.

평면을 45도 회전하면, 위 거리는 $|x_1 - x_2| + |y_1 - y_2|$ 형태의 L_1 거리 (맨하탄 거리)로 변환된다. 일반성을 잃지 않고 $x_1 \leq x_2, y_1 \leq y_2$ 라고 하자 (4방향으로 모두 회전시키면서 해 보면 된다). 우리는 고정된 (x_1, y_1) 에 대해서 $x_1 \leq x_2, y_1 \leq y_2$ 를 만족하며 $x_2 + y_2$ 를 최소화하는 점을 찾아야 한다. 이는 x 좌표에 대해 스위핑을 하고, y 좌표에 대해서 $x + y$ 의 최솟값을 저장하는 세그먼트 트리를 관리하면 된다. 시간 복잡도는 $O(N \log N)$ 이다.

부분문제 4 ($N \leq 50$)

부분문제 4에서 6까지는 정사각형을 찾는다고 생각하지 않고, 각 변의 길이가 w, h 일 때 $\max(w, h)$ 를 최소화 하는 직사각형을 찾는다고 생각한다. 직사각형은 x 축에 수직인 두 변 $x = x_s, x = x_e$, y 축에 수직인 두 변 $y = y_s, y = y_e$, 이렇게 총 4개의 정수로 표현할 수 있다. 직사각형의 4개의 변 중 하나에 점이 닿지 않는다면, 점이 닿을 때까지 해당 변을 안쪽으로 당기면서 $\max(w, h)$ 를 유지하거나 줄일 수 있다. 이는 우리가 고려해야 할 (x_s, x_e, y_s, y_e) 쌍의 후보가 $O(N^4)$ 개라는 것을 뜻한다: x_s, x_e 는 입력으로 주어진 $O(N)$ 개의 x 좌표 중 하나고, y 축에 대해서도 동일하기 때문이다. 고로 모든 가능한 $O(N^4)$ 개의 정사각형을 나열한 후, 이들 안에 들어있는 점들의 서로 다른 색이 K 개 인지를 배열에 마킹하는 식으로 확인하면 된다. 시간 복잡도는 $O(N^5)$ 이다.

부분문제 5 ($N \leq 150$)

x_s, x_e 를 고정시키고, $x_s \leq x \leq x_e$ 를 만족하지 않는 점들을 모두 무시해 주자. 이제 y 좌표만이 중요하니, 1차원에서 이 문제를 해결하면 된다. 점을 y 좌표 순으로 정렬한 후, y_s 를 고정해 주고 y_e 를 늘려주면서, 서로 다른 색이 K 개가 되면 답을 갱신하자. 1차원 문제가 $O(N^2)$ 에 풀리니 전체 문제는 $O(N^4)$ 에 풀린다.

부분문제 6 ($N \leq 600$)

부분문제 5와 동일하게 1차원 문제를 해결한다. y_s 를 고정해 주고 y_e 를 늘려주면서, 서로 다른 색이 K 개인 것이 확인되면 처음부터 시작하는 것이 아니라 y_s 를 한 칸 늘려준 후 (가장 왼쪽 점을 제거한 후) 다시 y_e 를 서로 다른 색이 K 개일 때까지 늘려주는 것을 반복한다. 이러한 방식을 Sliding Window 혹은 Two Pointers라고 하며, 점이 한 번 구간에 들어가고 한 번 나가니 $O(N)$ 에 1차원 문제가 해결된다. 정렬을 해야 한다고 생각할 수 있겠지만, 맨 처음에 점을 y 좌표 기준으로 한번만 정렬해 주면 굳이 매번 정렬할 필요가 없다. 전체 문제는 $O(N^3)$ 에 풀린다.

부분문제 7 ($N \leq 2500$)

길이 R 이하의 화려한 정사각형이 존재하는가? 라는 결정 문제로 문제를 변형해서 해결한다. 이 문제를 해결할 수 있다면, 최소의 R 을 이분 탐색으로 찾을 수 있다. R 을 고정할 경우, x_s, x_e 로 가능한 후보는 $O(N)$ 개가 된다. 왼쪽 변에 점이 닿는 경우가 N 개이고, 오른쪽 변에 점이 닿는 경우가 N 개이니 $2N$ 개만 시도해 보면 되기 때문이다: 둘 다 닿지 않는 경우는 앞과 비슷하게 왼쪽이나 오른쪽 점에 부딪힐 때까지 구간을 움직여 주면 된다. 이제 부분문제 6처럼 1차원 문제를 $O(N)$ 에 푼 후 그 답이 R 이하인지를 $2N$ 번 확인해 주면, 결정 문제가 $O(N^2)$ 에 풀리고, 전체 문제는 $O(N^2 \log X)$ 에 풀린다.

부분문제 8

부분문제 7과 동일한 결정 문제를 Plane sweeping을 사용하여 효율적으로 해결한다. 길이 R 의 x 좌표 구간 $[x_c, x_c + R]$ 를 왼쪽에서 오른쪽으로 움직이자. x_c 가 증가하는 과정에서 N 개의 점들은 $x_c = x_i - R$ 시점에 구간에 삽입되고, $x_c = x_i + 1$ 시점에 구간에서 제거된다. 문제를 해결하기 위해서는, 점들이 추가되고 제거될 때, 모든 K 개의 색의 점을 포함하는 길이 R 의 구간이 있는지를 빠르게 판별해야 한다.

$distinct[y]$ 를 $[y - R, y]$ 구간을 덮었을 때 얻을 수 있는 서로 다른 색의 개수라고 정의하자. $distinct[y] = K$ 인 x 가 존재한다면, 답이 존재한다고 판별할 수 있다. 점 하나가 추가되고 제거될 때 $distinct[y]$ 가 어떠한 식으로 변경되는지 살펴보자.

각 색에 대해서, 해당 색의 점들의 y 좌표들을 `std::multiset` 과 같은 자료 구조로 관리하자. y 좌표가 y_j 인 점을 추가할 때, 기본적으로 이 점이 새롭게 $distinct[i]$ 를 1 증가시킬 구간은 $[y_j, y_j + R]$ 이 된다. 하지만, 이미 y_j 의 직전에 있는 점이 합집합을 덮고 있어서 더하지 않아도 되는 구간, 직후에 있는 점에 가로막혀서 더하지 않아도 되는 구간들이 존재한다. 이 구간은 y_j 의 양 옆으로 인접한 점들의 좌표만 알면 케이스 처리로 구할 수 있다. 고로, `std::multiset`의 `lower_bound` 함수를 사용하면 새롭게 기여하게 되는 구간을 $O(\log N)$ 에 계산할 수 있다. 삭제에서도 역시, $distinct[i]$ 가 1 감소하는 구간을 아주 유사한 방식으로 계산할 수 있다.

이제 문제는 구간에 대해서 특정 수를 더하고, 전체 배열에 값이 K 인 원소가 있는지를 체크하는 구간 쿼리 문제가 된다. 배열의 특정 원소의 값은 항상 K 이하이므로, 세그먼트 트리에 Lazy propagation을 사용하여 해결할 수 있다. 결정 문제가 $O(N \log X)$ 에 해결되니, 전체 문제도 $O(N \log^2 X)$ 에 해결되어 만점을 받을 수 있다.